

Network Automation and Integrated Security Challenge

Laboratory 5

Student Practical Laboratory Manual

Course	CORE102 TCP IP Networking and Linux Network Tools
Instructor	Aminu Idris AMCPN
Maximum score	100 marks
Gradebook category	Practical Laboratories
Estimated duration	6 to 8 hours
Platform	Kali Linux or Ubuntu Linux in an authorised laboratory environment
Version	September 2026

Student name AGBO DANIEL AMEH
Registration number C1126OCO17757
Submission date 10/02/2026

CORE102 Laboratory 5 — Controlled Automation and Network Health Check

Part 1 — Automation Plan and Scope

Before writing any code, I created the working directory at `~/core102_lab5` and wrote down what each tool was allowed to do. The authorised subnet for this lab is `192.168.56.0/24`. The network inventory reads local state only and makes no changes. The ping sweep is limited to addresses `.1` through `.254` with a one-second probe timeout. The service checker accepts one host inside the subnet and probes a short list of ports with a one-second timeout per port. The PCAP summary opens an existing capture file and refuses anything without a `.pcap` or `.pcapng` extension. Every one of these limits exists so the tools cannot overreach the authorised scope.

Part 2 — Bash Network Inventory

My first attempt to write `network_inventory.sh` went wrong. I used nano with a heredoc and the terminal got stuck at a continuation prompt before anything was saved. I switched to writing the whole script in one block using `cat`, which worked cleanly.

The script opens with `set -euo pipefail`. The `-e` flag exits the moment any command returns non-zero. The `-u` flag treats an unset variable as an error rather than an empty string, which catches typos. The `pipefail` option makes a pipeline's exit status reflect any command that failed inside it, not just the last one. Together they mean the script fails fast and loud instead of producing a half-finished report.

A small section helper prints labelled dividers, and the main body collects `hostname`, `user`, `interfaces`, `addresses`, `routes`, `neighbours`, `listening sockets`, `resolver configuration` and `disk usage` into a timestamped file under the chosen output directory. I ran `bash -n` to validate syntax, then ran the

script twice — once with the default evidence directory and once with a custom evidence folder. The find command confirmed both reports existed with different timestamps.

Looking at the report, my Kali box has eth0 up at 192.168.56.102 with a /24 mask, only the local subnet route with no default gateway, which is correct for the host-only topology. SSH listens on port 22, and a stub resolver points at 192.168.1.1. Nothing on the system was modified.

The set -e option exits on any command failure, set -u treats unset variables as errors, and pipefail propagates pipeline failures. The default run wrote into the evidence folder and the custom run into a custom evidence folder. Both directories were created automatically and timestamped filenames prevented overwrites. I added a section before the disk space part that shows the default gateway and the path to 192.168.56.1, printing a message if no default route is configured. Every command in the script is a query using tools like ip, ss, cat and df. The only writes are creating the output directory and redirecting output to a file. Running the script repeatedly produces only new report files.

Part 3 — Controlled Bash Ping Sweep

The authorised_ping_sweep.sh script has one job above all others: refuse to run unless the argument is exactly 192.168.56. If it isn't, the script prints a rejection message and exits with status 2 before the loop starts. I tested this with 203.0.113 and it refused immediately — no packets left the machine. That gate is the difference between a training exercise and an unauthorised scan.

The authorised run walks through .1 to .254 on the 192.168.56 network, sending one ICMP echo per host with a one-second wait. Three hosts replied: 192.168.56.100 which is the VirtualBox host-only DHCP address, 192.168.56.101 which is my Ubuntu server, and 192.168.56.102 which is my own Kali client that answers its own address locally. That matches the expected inventory exactly. A host that doesn't reply might still be online — its firewall could be dropping echo requests, the kernel could be rate-limiting replies, or a router on the path could be filtering ICMP.

I extended the script with a Started timestamp, a Finished timestamp and a running count that prints the total number of responsive hosts at the end. The scope gate stayed untouched.

The rejected prefix run printed a scope rejection message mentioning the authorised prefix and exited with status 2 — no packets were sent outside the authorised range. Three hosts responded, matching the expected inventory. A missing ping reply only proves no echo arrived within the timeout; it doesn't prove the host is offline. I added start and finish timestamps plus a responsive count while keeping the prefix gate unchanged.

Part 4 — Python Service Checker

This checker taught me how much more careful Python has to be about scope. Three checks run before any socket opens: the argument count must be exactly one host, the input must be a valid IPv4 address which the ipaddress module rejects if invalid, and the address must fall inside 192.168.56.0/24. Each failure has its own exit code — 1 for wrong arguments, 2 for a bad format, 3 for an out-of-scope address — so a caller can tell the difference between a typo and an attempt to scan outside the sandbox.

I tested all three failure paths first. No argument gave the usage line. Passing "hello" gave an invalid IPv4 address error. Passing 8.8.8.8 — a perfectly valid address — was still refused with a scope rejection message mentioning the authorised network and exit status 3.

Then I ran the checker against 192.168.56.101. Only port 22 came back OPEN — everything else was closed or filtered. I started the Python HTTP server on Ubuntu binding to all interfaces on port 8080 and re-added the UFW rule for 8080 from the Kali client. Re-running the check flipped 8080 from CLOSED OR FILTERED to OPEN. Then I stopped the server with pkill and ran the check again — 8080 went back to CLOSED OR FILTERED while 22 stayed OPEN. Four runs in one screenshot tell the whole story.

The three validation tests produced exit codes 1, 2 and 3 respectively. Port 8080 changed from CLOSED OR FILTERED to OPEN after the server started and the UFW rule was re-added. Stopping the service flipped it back, proving live state. I added an optional port argument, validated as an integer between 1 and 65535, with the address-scope check still running first.

Part 5 — Offline PCAP Summary

Neither VM had internet access, so pip install scapy failed with a DNS resolution error. Rather than give up on Part 5, I rewrote pcap_summary.py using only Python's standard library. It reads the 24-byte global header, detects endianness from the magic number, then loops through per-packet records. For each packet it strips the link-layer header based on linktype supporting Ethernet, Linux SLL, or BSD loopback, parses the IPv4 header, and classifies by protocol number. For TCP it extracts both ports.

I generated a real capture by running the Python HTTP server on Ubuntu binding to all interfaces on port 8080, then using tcpdump on the loopback interface to write a pcap file filtering for tcp port 8080, followed by two curl requests to localhost on port 8080. The summary was clean: 8 packets, all TCP, 16 endpoint references to 127.0.0.1, and TCP port 8080 appearing 8 times. The four client ephemeral ports each appeared twice.

The tcpdump report showed 8 packets captured and my script counted 8 from the file — the same number. Both source and destination ports are counted so server and client ends can be distinguished. I added a DNS counter for port 53 in both the TCP and UDP branches. A limitation is that port-based classification can be fooled by services on non-standard ports or by tunnelled protocols, since it never inspects the payload.

Part 6 — Integrated Security Challenge

The Ubuntu interface enp0s3 was up at 192.168.56.101 with a /24 mask, and Kali's eth0 was up at 192.168.56.102 with a /24 mask. Neither host has a default gateway, matching the host-only topology.

Running ufw status verbose on Ubuntu returned Default deny for incoming and allow for outgoing. Numbered rules allow SSH from the 192.168.56.0/24 subnet, allow 8080 from 192.168.56.102, and deny Telnet on port 23.

From Kali I connected using ssh with the core102_ed25519 identity file to santi at 192.168.56.101. It prompted for the key passphrase, not the account password, and logged in as santi on the Ubuntu machine. The server journal recorded an accepted public key with fingerprint e4NCcgi0GzU41g+jRHxtrCn80xZwc4sUDT301D/1SgE. Password authentication is disabled in the hardening drop-in file.

I copied network_inventory.sh to Ubuntu with scp and it ran cleanly, saving a timestamped evidence file. The ping sweep on Kali found three hosts — .100, .101, .102 — across three identical runs.

The service checker against 192.168.56.101 reported 22 OPEN and 53, 80, 443, 8080 CLOSED OR FILTERED. Only sshd is exposed on the network interface.

On Ubuntu I captured SSH traffic using tcpdump on enp0s3 writing to a pcap file filtering for tcp port 22 and host 192.168.56.102. After one SSH command from Kali and exit, 629 packets were captured with 270 matching the filter. The summary showed TCP port 22 dominating with endpoints only .101 and .102, and nothing readable after the handshake.

Starting the Python web server on Ubuntu with the UFW rule present made 8080 show OPEN. Stopping the server with pkill flipped it back to CLOSED OR FILTERED while 22 stayed OPEN. The two runs at 14:51:58 and 14:52:44 prove the change is real.

The 192.168.56.0/24 host-only network has Ubuntu at .101 and Kali at .102. Firewall default-deny incoming; SSH allowed only from the subnet; Telnet denied; web test port scoped to the authorised client. SSH key authentication works with the Ed25519 key. Only sshd is network-reachable — DNS and NTP listen on loopback only. Ping sweep found exactly the three expected hosts. SSH capture verified encryption after the handshake. Service check flipped OPEN and CLOSED when the web server started and stopped.

Unresolved issues: no internet on either VM meant Scapy couldn't be installed, so the pcap summary uses a standard-library parser with equivalent TCP/UDP/ICMP counting but no deeper dissection. The VirtualBox host-only adapter also loses IPv4 addresses on reboot.

For cleanup I stopped the web server, removed the 8080 UFW rule, and deleted the core102lab0 dummy interface. The SSH key pair is retained at ~/.ssh/core102_ed25519 pending instructor confirmation.

Part 7 — Code Review Checklist

Every script begins with an interpreter line. Bash scripts pass `bash -n` syntax checking and Python scripts compile with `py_compile`. Inputs are validated before any network action. The authorised scope is explicit in code and output. Network operations use a timeout or bounded packet count. Output includes a timestamp and identifies the target. Error messages explain why the program stopped. No passwords, private keys or tokens appear in any file.

Reflection Questions

1. Which validation control most effectively prevented an out-of-scope action?*

The explicit scope gate. In the ping sweep, the prefix check runs before the loop, so nothing leaves the host unless the argument matches exactly. In the service checker, the network membership check runs before any socket is opened. These gates physically prevent the tools from touching anything outside the lab.

2. What can a service checker conclude, and what can it not conclude, from a timeout?*

It can conclude that no TCP handshake completed within the timeout window. It cannot conclude that the port is closed, the service is down, or the host is unreachable — a timeout could mean the firewall dropped the packet, the service is slow, or the packet was filtered upstream. Only a successful handshake gives a definitive positive answer.

3. How did automation improve repeatability compared with running commands manually?*

Every run produced an identical output structure with the same timestamp format, scope validation and failure behaviour. Three separate ping sweeps all reported the same three hosts with the same layout. Manual runs would drift in format, scope enforcement and timing, making comparisons between runs unreliable.

4. Which additional control would you add before these scripts could support a larger authorised environment?

Centralised logging and a signed allow-list of targets. Right now each script writes local files that could be lost or tampered with. For a bigger environment I would route every sweep and check to a central syslog server with operator identity and timestamps, and hold target scopes in a signed configuration file rather than a hard-coded constant. Rate limiting between probes would also protect large subnets from bursts.

Final Checklist

I worked only in an authorised laboratory environment. Full name and registration number will be added to the report header before submission. Every required command and result is visible in the evidence. Results are explained, not just shown. Passwords, private keys and personal information have been stripped. The PDF and ZIP have been opened to confirm completeness.

The submission package includes the report PDF and a ZIP file containing the four scripts, their generated text reports, and the authorised PCAP. The slash in the registration number has been replaced with a dash or underscore because filenames cannot contain slashes.